

玩转 Hermes Agent, 构建可落地的多智能体系统

虾王殿下

2026-04-17

目录

《玩转 Hermes Agent，构建可落地的多智能体系统》

图书定位

目标读者

写作风格

全书目标

第一章 为什么是 Hermes Agent

本章小结

延伸阅读

第二章 Hermes Agent 的核心概念

2.1 什么是 Agent

2.2 Tool、Memory、Workflow 分别是什么

2.3 任务分解与角色设计

2.4 单智能体与多智能体的差异

2.5 一个最小 Hermes Agent 系统长什么样

本章小结

延伸阅读

第三章 快速搭建第一个 Hermes Agent

3.1 环境准备，先别急着开工

3.2 第一个 Agent，任务一定要选对

3.3 先定义目标，再定义行为

3.4 一个最小 Hermes Agent 应该包含什么

3.5 第一次运行时，你真正要看什么

3.6 第一次搭建最常见的坑

本章小结

延伸阅读

第四章 提示词设计与行为控制

4.1 Prompt 的作用，到底是什么

4.2 System Prompt，决定的是底层人格和边界

4.3 用户提示词，不是重复系统规则

4.4 行为控制，真正靠的是“约束清晰”，不是“语气严厉”

4.5 如何让 Agent 少跑偏

4.6 Prompt 不是万能的，别拿它替代系统设计

本章小结

延伸阅读

第五章 工具接入，让 Agent 真正干活

5.1 没有工具的 Agent，为什么总差一口气

5.2 Tool 的本质，不是插件，而是能力边界

5.3 常见工具类型，可以怎么分类

5.4 第一次接工具，为什么一定要克制

5.5 工具调用什么时候该发生

5.6 工具失败，不要让 Agent 硬装成功

5.7 从“能调用工具”到“工具链可落地”

本章小结

延伸阅读

第六章 记忆与上下文管理

6.1 上下文窗口，不等于记忆系统

6.2 记忆的本质，是“任务连续性基础设施”

6.3 短期记忆，服务当前任务过程

6.4 长期记忆，服务跨任务复用

6.5 不是所有信息都该记住

6.6 好的记忆系统，一定要支持检索和回收

6.7 记忆污染，是 Agent 系统最隐蔽的慢性病

本章小结

延伸阅读

第七章 多智能体协作系统设计

7.1 为什么需要多智能体

7.2 什么时候该拆，不要太早，也不要太晚

7.3 多智能体系统里，最常见的几种角色

7.4 串行、并行、路由，这三种协作模式有什么区别

7.5 多智能体真正难的，不是拆，而是协同

7.6 多智能体系统最容易翻车的几个坑

本章小结

延伸阅读

第八章 实战案例，从 0 到 1 搭一个业务系统

8.1 先选一个真实、完整、边界清晰的业务场景

8.2 需求分析，先别急着搭

8.3 任务拆解，把“大目标”拆成“稳定模块”

8.4 架构设计，把模块映射成 Agent

8.5 工具与记忆怎么接入

8.6 Workflow 设计，把系统真正跑起来

8.7 调试、修正与上线前检查

8.8 复盘，这个案例真正说明了什么

本章小结

延伸阅读

第九章 性能优化与稳定性治理

9.1 成本控制，不是小气，是系统理性

9.2 响应速度，慢不是一定更稳

9.3 失败重试与异常恢复，决定系统有没有韧性

9.4 监控、日志与评估，没有这些就别谈优化

9.5 用数据驱动迭代，不要靠感觉修系统

本章小结

延伸阅读

第十章 从可用到可交付

10.1 演示，不是炫功能，而是讲清价值闭环

10.2 交付，不只是给代码，更是给使用方式

10.3 团队协作里，Agent 系统也需要分工

10.4 持续迭代，别把 Agent 系统做成一次性交付品

10.5 Hermes Agent 的未来机会

本章小结

附录 A 常见术语表

附录 B Hermes Agent 常见问题排查清单

附录 C 推荐实践清单

全书结语

《玩转 Hermes Agent，构建可落地的多智能体系统》



图书定位



一本面向 AI Agent 开发者、产品经理与自动化 workflow 设计者的实战指南。本书聚焦 Hermes Agent 的真实落地，不停留在概念介绍，而是围绕“怎么搭、怎么跑、怎么稳、怎么交付”展开，帮助读者从单智能体走向多智能体系统设计与实战部署。

目标读者



- AI Agent 开发者
- 智能体产品经理
- 自动化流程搭建者
- 希望把大模型能力真正落地到业务场景的人

写作风格



- 中文讲解
- 实战优先
- 少空话，重案例、重结构、重避坑
- 默认风格为技术实战派，兼顾体系感与可操作性

全书目标



读完本书后，读者应能够：

1. 理解 Hermes Agent 的核心设计思想
2. 独立搭建一个基础可运行 Agent
3. 为复杂任务设计多智能体协作结构
4. 接入工具、记忆和工作流能力
5. 对 Agent 系统进行调试、优化与交付

第一章 为什么是 Hermes Agent



如果你这两年一直在看大模型圈子的变化，你会发现一个很明显的趋势，大家讨论的重点，正在从“模型本身有多强”，转向“模型到底怎么真正干活”。

一开始，很多人对大模型的期待很简单，无非是把它当成一个更聪明的问答工具。你问一句，它答一句。写文案、改润色、查资料、生成代码，这种模式确实已经很强了，也让无数人第一次真切感受到 AI 的生产力价值。

但问题也很快暴露出来了。

当任务开始变复杂时，单次问答式交互就会迅速碰到天花板。比如你希望 AI 帮你完成一份完整的行业研究，不只是回答一个问题，而是自己拆任务、查资料、比对信息、整理结论、输出结构化结果。再比如你希望它不只是写一段代码，而是能理解需求、规划模块、调用工具、修改文件、测试结果、持续修正。到了这一步，传统的“一问一答”已经不够用了。

真正的业务任务，往往不是一句提示词就能解决的。

它们有上下文，有状态，有过程，有依赖，有失败重试，有不同角色之间的协作，还经常需要访问工具、文件、网页、数据库甚至其他系统。也就是说，问题已经不再是“模型会不会回答”，而是“模型能不能像一个系统那样持续执行任务”。

这就是 AI Agent 概念迅速走热的根本原因。

所谓 Agent，你可以先把它理解成一种“有目标、能行动、会调用资源、能根据反馈继续推进任务”的 AI 执行体。它不只是一个等你发问的回答机器，而更像一个能够围绕目标连续行动的数字执行者。它能接收任务，理解目标，决定下一步做什么，调用合适的工具，在必要时保存上下文，并根据结果继续调整自己的行为。

从这个角度看，Agent 并不是大模型的替代品，而是大模型能力的组织方式升级。模型负责理解与生成，Agent 负责推进与执行。模型像大脑的一部分，Agent 更像把大脑、手脚、记忆和流程组织起来之后形成的行动系统。

这也是为什么越来越多团队开始从“Prompt Engineering”转向“Agent Engineering”。前者关心的是一句提示词怎么写得更好，后者关心的是如何把模型、工具、上下文、流程、记忆和协作方式组合成一个真正可运行、可复用、可优化的系统。

而 Hermes Agent，正是在这样的背景下变得重要起来。

Hermes Agent 的价值，不在于它只是又一个“能调用大模型的壳”，而在于它把很多零散的能力，组织成了适合真实业务落地的执行框架。对于开发者、产品经理和自动化 workflow 设计者来说，它提供的不是一个单点能力，而是一种更接近工程现实的系统化方法。

这种方法至少解决了四类核心问题。

第一，它让任务执行从“对话式”走向“流程式”。你不再只是和模型来回聊，而是可以围绕任务目标去定义角色、行为、工具、规则和执行顺序。这样一来，AI 的能力就不只是输出一段文字，而是开始真正参与任务流程本身。

第二，它让工具使用变得可组织。很多人都知道，大模型一旦接入工具，能力会明显增强。但真正的难点从来不是“能不能接”，而是“怎么接得稳、接得安全、接得可控”。Hermes Agent 的价值之一，就是帮助你把工具调用纳入一个更可管理的结构里，让 Agent 不只是能调用工具，而是知道什么时候调、调哪个、失败了怎么办。

第三，它让上下文和记忆不再只是临时拼接。复杂任务一定依赖上下文，而上下文如果完全靠人工不断粘贴补充，系统很快就会失控。Hermes Agent 更适合处理需要持续状态、连续任务和多轮推进的场景，因为它不是把每次执行都当成孤立事件，而是允许你站在系统层面思考信息如何保存、如何继承、如何更新。

第四，它为多智能体协作提供了基础土壤。单智能体有它的优势，简单、直接、启动成本低。但一旦任务变长、变深、变复杂，一个 Agent 往往既要理解需求、又要查资料、又要规划流程、又要执行细节，最后什么都沾一点，什么都不够稳。多智能体系统的意义，就在于把不同任务拆给不同角色，让系统更像一个团队，而不是一个孤军奋战的超人。Hermes Agent 的结构，使这种拆分和协作更容易落地。

当然，说到这里，也不能把 Hermes Agent 神化。

它不是万能药，不是装上就能自动替你把所有业务跑起来。它仍然需要明确的目标、合理的流程设计、稳定的工具链、可验证的执行结果，以及持续的调试和优化。很多人做 Agent 项目失败，不是因为模型不够强，而是因为系统设计太随意，边界不清，依赖混乱，出了问题无法排查。

所以，本书不会把重点放在“吹 Hermes Agent 有多厉害”这种空话上。真正有价值的问题是：

它到底适合解决什么问题？它的系统结构应该怎么理解？一个可运行的 Hermes Agent 到底该怎么搭？什么时候该用单智能体，什么时候该拆成多智能体？工具、记忆、 workflow、评估，这些模块到底该怎么组合？以及，最重要的，怎么把一个“看起来很聪明”的 Demo，变成一个真正能交付的系统？

这正是本书要回答的核心问题。

在接下来的章节里，我们不会停留在概念层面，而是会一步步走进 Hermes Agent 的实际构建过程。你会看到它的核心组成部分，学会如何搭建第一个可运行的 Agent，理解如何通过提示词和规则设计来约束行为，掌握工具接入与记忆管理的方法，并最终把这些能力拼成一个真实可用的多智能体系统。

如果你之前只是把 AI 当成一个更聪明的聊天框，那么这本书想带你迈出下一步，让它从“会回答”变成“会执行”，从“能演示”变成“能落地”。

这也是为什么，第一章我们讨论的不是某个具体命令，不是某个配置字段，而是先回答一个更根本的问题：

为什么是 Hermes Agent。

本章小结



本章的重点，不是教你立刻写出一个 Hermes Agent，而是先帮你建立一个更关键的判断：为什么今天我们需要从“大模型问答”走向“Agent 系统设计”。

我们看到，单次提示词交互虽然已经足够强大，但一旦任务进入真实业务场景，问题就不再只是“模型能不能回答”，而变成“系统能不能持续执行”。任务需要分解，流程需要组织，工具需要调用，信息需要保存，结果需要校验，复杂任务还需要多个角色协同推进。在这种情况下，单纯依赖对话式交互，很难稳定支撑真实落地。

Hermes Agent 的价值，就在于它提供了一种更贴近工程现实的组织方式。它不是单纯把模型接进来，而是尝试把模型、工具、记忆、流程和协作结构组合成一个可执行、可扩展、可优化的系统。也正因为如此，它更适合承担那些有状态、有步骤、有依赖关系的任务。

但我们也强调了，Hermes Agent 不是万能药。它的效果高度依赖系统设计本身。目标是否清晰，角色是否合理，工具是否稳定，边界是否明确，决定了一个 Agent 系统究竟是生产力工具，还是一个看起来热闹却不可靠的展示品。

所以，从这一章开始，你应该先建立一个基本认知：

- Hermes Agent 不是一个聊天工具的替代品
- 它是一种组织 AI 执行能力的系统方法
- 真正的重点不在“会不会调用模型”，而在“能不能把任务稳定跑完”

接下来的内容，我们就会从这个认知出发，逐步进入 Hermes Agent 的核心结构与搭建实践。

延伸阅读



建议这一章后面配一个“延伸阅读”小节，风格别太学术，保持实战向。可以先放这几类：

1. 先理解 Agent 这个概念到底和传统调用差在哪

- 大模型问答式交互 vs Agent 持续执行
- Workflow、Assistant、Agent 三者的区别
- 单智能体与多智能体的适用边界

2. 看几个真实业务里适合 Agent 的任务类型

- 资料搜集与结构化整理
- 自动化运营流程
- 多步骤内容生成
- 研发辅助与代码工作流
- 企业内部知识助手与任务执行系统

3. 关注系统落地，而不是只看 Demo

- 工具调用稳定性
- 状态与记忆管理
- 错误处理与回退机制
- 成本、速度与可靠性的平衡

4. 带着问题进入下一章

读者在进入第二章前，脑子里最好先带着这几个问题：

- 一个 Hermes Agent 最小可运行单元到底包含什么
- Tool、Memory、Workflow 在系统里分别扮演什么角色
- 为什么同样是“AI 自动执行”，有的系统稳定，有的系统很快失控

第二章 Hermes Agent 的核心概念



在真正开始搭建一个 Hermes Agent 之前，我们得先把几个最核心的概念钉牢。否则后面一上手，很容易陷入一种典型混乱，看起来好像搭了很多模块，实际上自己都说不清系统到底是怎么运作的，出了问题也不知道该改哪一层。

很多人第一次接触 Agent 系统时，最容易犯的错误，就是把所有东西混成一团。模型是模型，提示词是提示词，工具是工具，记忆是记忆，流程是流程，但一旦进入项目里，这些概念常常被混着用。结果就是，一出问题就开始盲调，要么改 prompt，要么换模型，要么加规则，要么堆工具，最后系统越来越复杂，稳定性却越来越差。

所以这一章的目标很明确，不是讲技巧，而是帮你建立一套足够清晰的系统认知。你得先知道一个 Hermes Agent 系统里，到底有哪些核心部件，它们分别负责什么，它们之间又是怎么协同起来的。

2.1 什么是 Agent



先说最核心的概念，Agent 到底是什么。

如果用最朴素的话来说，Agent 就是一个围绕目标持续推进任务的 AI 执行体。它和普通大模型对话最大的不同，不在于“更聪明”，而在于“更能干活”。它不是只对当前输入做一次回应，而是会围绕某个目标，决定下一步动作，调用外部资源，处理执行结果，并继续向前推进。

这里有三个关键词特别重要：

- 有目标
- 能行动
- 会持续推进

这三点缺一不可。

如果只有回答能力，没有明确目标，它更像一个聊天模型。如果只有目标，没有行动能力，它只是一个会分析的顾问。如果能行动，但不会基于结果继续推进，那也只是一次性脚本，不算真正意义上的 Agent。

所以，Agent 的本质，不是“一个更复杂的 prompt”，而是“一个具备任务推进能力的执行单元”。

在 Hermes Agent 的语境下，你可以把 Agent 理解成一个带有行为约束、工具能力、上下文状态和执行逻辑的任务执行角色。它能接任务，理解目标，决定行为，并在限定规则内完成事情。

2.2 Tool、Memory、Workflow 分别是什么

理解 Agent 之后，接下来就得看它靠什么完成任务。

最常见的三个支撑组件，就是：

- Tool
- Memory
- Workflow

这三个词你会在很多 Agent 框架里看到，但真正难的不是背定义，而是理解它们在系统里的职责边界。

Tool: Agent 的手脚

Tool 可以理解成 Agent 用来接触外部世界的能力接口。

大模型本身再强，如果只能在上下文里输出文字，那它本质上还是“会说不会做”。而 Tool 的作用，就是把“说”变成“做”。它让 Agent 可以访问文件、调用 API、搜索网页、读取数据库、发送消息、执行脚本，甚至和其他系统交互。

所以 Tool 的核心意义，不是增加花样，而是让 Agent 具备外部行动能力。

你可以把模型看作大脑，把 Tool 看作手脚。没有 Tool，Agent 只能纸上谈兵。Tool 越稳定、越清晰、越可控，Agent 的落地能力就越强。

但这里要注意，Tool 不是越多越好。工具多，不代表系统强，反而可能让 Agent 频繁误用、乱用、滥用。真正有效的做法，是围绕具体任务场景设计工具集合，让 Agent 在需要的时候调用正确工具，而不是给它一个无边无际的玩具箱。

Memory: Agent 的记忆系统

Memory 的作用，是让 Agent 不是每次都从零开始。

很多初学者会误以为，只要上下文窗口够大，记忆问题就自然解决了。其实不是。上下文窗口解决的是“当前能看到多少信息”，而 Memory 解决的是“哪些信息应该被保留、如何被保留、什么时候被再次调用”。

这两者不是一回事。

如果没有记忆，Agent 每轮都像失忆一样。如果记忆设计混乱，Agent 又会被旧信息污染。所以 Memory 的关键，不是单纯“存更多”，而是“存得对、取得到、用得稳”。

通常来说，记忆至少可以分成两类：

- 短期记忆：服务当前任务过程，帮助 Agent 记住刚刚发生了什么
- 长期记忆：服务跨任务复用，保存偏好、规则、经验、历史结果等

在 Hermes Agent 这种系统里，Memory 更像任务持续性的基础设施。它帮助系统维持状态，让多步骤任务不会每次都断片，也让系统有机会随着使用逐渐变得更贴合真实场景。

Workflow: Agent 的行动路径

如果说 Tool 是手脚，Memory 是记忆，那么 Workflow 更像路线图。

Workflow 的作用，是规定任务该怎么推进。不是所有任务都适合让 Agent 完全自由发挥。很多真实业务里，更有效的方式恰恰是给它一个清晰的执行路径，让它在既定流程中完成判断和动作。

比如一个内容生产任务，可能天然就包含：

1. 收集资料
2. 提取要点
3. 生成大纲
4. 起草内容
5. 审校润色
6. 输出结果

这个顺序本身，就是 Workflow。

Workflow 不是为了限制智能，而是为了减少混乱。特别是在需要稳定性、可重复性和可验证性的业务场景里，没有流程约束的 Agent，往往跑着跑着就飘了。

所以你可以这样理解：

- Tool 决定能做什么
- Memory 决定记住什么

- Workflow 决定先做什么、后做什么

这三者加在一起，才让 Agent 从一个“会回答的模型”变成一个“会执行的系统角色”。

2.3 任务分解与角色设计

Agent 系统一旦进入真实任务，几乎都会碰到一个问题，任务太大，没法一口吃掉。

比如“帮我做一份市场研究报告”，这句话看起来是一条任务，实际上里面至少包含信息搜集、资料判断、结构整理、观点形成、内容写作、结果检查等多个不同类型的子任务。如果把这些全压给一个 Agent，它大概率不是做不完，而是做不稳。

这时就需要任务分解。

任务分解的目的，不是把事情拆得越碎越高级，而是把一个复杂目标，拆成若干边界清晰、易于执行、便于验证的模块。只有这样，系统才容易稳定。

通常来说，好的任务分解要满足三个标准：

- 每个子任务目标明确
- 每个子任务产出可验证
- 子任务之间关系清楚

在此基础上，角色设计就自然出现了。

所谓角色设计，就是给不同执行单元定义不同职责。比如：

- 主控 Agent 负责理解总目标和调度流程
- 研究 Agent 负责搜集与整理信息
- 写作 Agent 负责内容生成
- 审校 Agent 负责检查质量与一致性

这样做的好处很明显。每个 Agent 不需要什么都懂，但它要把自己的职责做稳。系统不再依赖一个“全能选手”，而是靠一组边界清晰的角色完成协作。

当然，角色并不是越多越高级。角色过多只会增加调度复杂度和沟通损耗。合理的原则是，只有当任务复杂到一个 Agent 难以稳定承担时，才值得拆角色。

2.4 单智能体与多智能体的差异



很多人一聊 Agent，就容易兴奋地往多智能体上冲，仿佛不搞几个 Agent 互相对话，就不够先进。其实这是个很常见的误区。

单智能体并不落后，多智能体也不天然更好。

单智能体最大的优势是简单。它适合目标清晰、流程较短、依赖较少的任务。配置简单，调试方便，成本更低，出问题也更容易定位。对于很多实际业务来说，一个设计得足够好的单 Agent，已经能解决 70% 甚至 80% 的问题。

多智能体的价值，不在于“看起来更酷”，而在于它适合处理那些天然需要分工协作的复杂任务。

什么时候该考虑多智能体？通常有几个信号：

- 一个任务同时包含多种能力类型
- 单个 Agent 上下文负担过大
- 任务链条较长，需要阶段性产出
- 不同步骤需要不同风格或不同判断标准
- 系统需要更好的扩展性和可维护性

换句话说，多智能体不是炫技，而是一种系统拆分方法。它的核心收益来自分工，而它最大的代价则是协调。

因为一旦进入多 Agent 系统，新的问题就会出现：

- 谁来调度
- 信息怎么传递
- 状态怎么同步
- 出错后谁负责回滚

- 冲突结果怎么裁决

所以，多智能体系统真正难的，从来不是“多几个 Agent”，而是“如何让多个 Agent 不互相添乱”。

2.5 一个最小 Hermes Agent 系统长什么样



理解完前面的概念，我们最后要落到一个最关键的问题：

一个最小可运行的 Hermes Agent 系统，到底长什么样？

答案其实比很多人想象得简单。最小系统不需要一上来就堆满所有模块，它只需要四个最基础的组成：

1. 一个清晰的目标
2. 一个可被约束的 Agent
3. 一组必要的工具
4. 一个基本可执行的流程

比如，一个最简单的内容整理型 Hermes Agent，可以是这样：

- 目标：把一篇长文整理成结构化摘要
- Agent：负责理解任务、调用工具、组织输出
- Tool：读取文件、搜索资料、写入结果
- Workflow：读取原文 提取重点 生成摘要 输出 Markdown

这已经是一个完整的最小闭环。

如果再进一步，你可以加入：

- Memory，让它记住用户偏好
- Review 节点，让它在输出前自检
- 子 Agent，让研究和写作分开

但这些都是最小系统跑通之后，再逐步增加的能力。真正成熟的做法，从来不是一开始就做大，而是先搭出一个最小可运行单元，然后一点点把系统能力扩进去。

这也是本书后面会一直坚持的原则：

先跑通，再做强；先清晰，再做复杂。

很多 Agent 项目不是死在能力不够，而是死在一开始就堆太多。模型想要最强，工具想接最多，角色想拆最细，流程想设计最全，最后结果往往是系统看起来很豪华，实际一跑全是噪音。

Hermes Agent 真正有价值的地方，不在于它允许你把系统做得多复杂，而在于它允许你从一个清晰的小系统出发，逐步长成一个真正可落地的系统。

本章小结



这一章，我们没有急着进入具体操作，而是先把 Hermes Agent 的核心概念拆清楚了。

最重要的结论有四个：

- Agent 的本质，是围绕目标持续推进任务的执行体
- Tool 是手脚，Memory 是记忆，Workflow 是行动路径
- 复杂任务必须经过合理分解，必要时再进行角色设计
- 多智能体不是目的，稳定协作才是目的

如果说第一章回答的是“为什么值得学 Hermes Agent”，那么这一章回答的就是“一个 Hermes Agent 系统到底是由什么构成的”。

只有把这层想清楚，后面真正开始搭建的时候，你才不会陷入“到处加模块、到处试配置、出了问题全靠猜”的混乱状态。

延伸阅读



读完这一章，建议带着下面几个问题进入下一章：

- 一个 Hermes Agent 真正启动起来时，最先要配置的是什么
- 第一个可运行 Agent 应该尽量简单到什么程度
- 在实际搭建时，哪些东西该先做，哪些东西先不要急着加
- 如何判断当前任务适合单智能体，还是已经需要多智能体拆分

第三章 快速搭建第一个 Hermes Agent



前两章我们一直在讲概念，讲 Agent 是什么，讲为什么要从模型调用走向系统设计，也讲了 Tool、Memory、Workflow 这些核心部件的分工。到这一步，很多读者脑子里通常会冒出一个问题：

说了这么多，到底怎么搭一个真的能跑的 Hermes Agent？

这一章，就来解决这个问题。

不过在正式开始之前，本王先说一句很重要的话。第一次搭 Hermes Agent，目标不是做一个“最强”的系统，而是做一个“最小但完整”的系统。也就是说，它不需要一开始就支持复杂工具链，不需要先把多智能体架构全铺开，也不需要一口气把记忆、评估、监控、工作流都堆满。

第一次搭建的唯一目标只有一个：

让一个 Hermes Agent 真正跑起来，并形成可观察、可验证、可继续迭代的最小闭环。

很多人第一次做 Agent 项目失败，不是因为不会写提示词，也不是因为模型太弱，而是因为一上来就想做得太大。结果系统结构没理顺，执行链路没打通，问题一出就不知道卡在哪。最后明明花了很多时间，却始终没有一个稳定的起点。

所以这章我们坚持一个原则：

先搭最小闭环，再谈复杂能力。

3.1 环境准备，先别急着开工



要搭一个 Hermes Agent，第一步当然不是写 prompt，而是先把运行环境准备好。

一个最基本的 Hermes Agent 环境，至少要有以下几类东西：

- 一个可用的大模型接口
- Hermes Agent 的运行框架或宿主环境
- 基础工具调用能力
- 日志或结果观察方式
- 必要的配置文件

很多人最容易忽视的是最后两项。模型和框架装好了，就觉得可以开干了，结果一跑起来根本看不清 Agent 到底做了什么。它是没理解任务，还是没选对工具，还是中途失败了，完全说不出来。这种搭建方式，后面一定越做越乱。

所以在开始之前，你至少要确认三件事：

第一，模型调用是通的

别一上来就怀疑 Agent。先确认底层模型本身能正常调用，输入输出稳定，认证信息没问题，响应时间在可接受范围内。

如果连模型层都没通，后面所有问题都会被混在一起，调起来像在海里捞针。

第二，最基础的工具能力可用

哪怕你只想先做一个最简单的 Agent，也最好准备一两个最基础的工具，比如：

- 读取文本
- 写入结果
- 调用简单搜索
- 获取本地文件内容

因为一旦 Agent 完全没有工具，它就只能在上下文里原地打转。而第一次搭建时，最好的方式恰恰是让它做一个很小的、但真实可执行的任务。

第三，有最基本的可观察性

所谓可观察性，不一定非得是很重的监控系统。第一次搭建时，只要满足下面几个要求就够了：

- 能看到输入任务
- 能看到 Agent 的输出
- 能看到它是否调用了工具
- 能看到大致执行流程
- 出错时能看到报错位置

这一步非常重要。第一次搭 Agent，调试能力比功能数量更重要。你宁可功能少一点，也要保证系统出了问题你知道问题在哪。

3.2 第一个 Agent，任务一定要选对

很多人第一次练手时，最容易做错的一件事，就是任务选太大。

比如一上来就想做一个“全自动行业研究智能体”，或者“全流程内容生成系统”，甚至“企业级多智能体助手平台”。这些想法听着很猛，但对于第一次搭建来说，基本等于给自己挖坑。

因为第一次搭建最需要的，不是复杂任务，而是明确反馈。

最适合练手的任务，应该满足四个条件：

- 目标明确
- 步骤不多
- 输出可验证
- 失败后容易定位问题

比如下面这类任务就很适合：

- 读取一篇文章，输出结构化摘要
- 根据一个需求描述，生成任务拆解清单
- 读取一个文档目录，生成内容提纲
- 根据一组输入信息，整理成固定格式结果

这些任务看起来简单，但足够真实，而且非常适合验证 Agent 的最小能力闭环。

所以第一次搭 Hermes Agent，任务不要选“最酷的”，要选“最容易验证的”。

本章后面的示例，我们就假设你要搭一个最简单的内容整理型 Agent，它的目标是：

读取输入文本，提取关键信息，并输出结构化摘要。

这个任务足够小，但已经完整包含了 Agent 系统的几个关键环节：

- 接收任务
- 理解目标
- 调用输入资源
- 组织输出
- 返回结果

换句话说，它已经不是“纯聊天”，而是一个最小的执行闭环。

3.3 先定义目标，再定义行为



很多初学者写 Agent，第一步就是先写一大段 prompt。这很常见，但顺序其实反了。

正确顺序应该是：

1. 先定义目标
2. 再定义边界
3. 再定义行为
4. 最后才是 prompt 文案

因为提示词本质上只是系统约束的一种表达方式，而不是系统设计本身。

比如，我们现在这个最小 Agent 的目标是：

将输入文本整理为结构化摘要，输出包含主题、核心观点、关键事实和结论。

接下来，就要给它补上行为边界。也就是告诉它：

- 它要完成什么
- 它不需要完成什么
- 它输出应该长什么样
- 它遇到信息不足时该怎么处理

一个最小行为定义，可以长这样：

- 输入是一段文本
- 输出是固定结构的摘要
- 不编造原文中没有的信息
- 如果信息不足，要明确说明

- 输出尽量简洁、清晰、结构化

你会发现，到这一步，其实 prompt 已经呼之欲出了。因为系统设计已经先完成了，文字只是把规则说清楚。

这比一上来就“写一段很厉害的提示词”稳得多。因为后者靠感觉，前者靠结构。

3.4 一个最小 Hermes Agent 应该包含什么

当目标和行为边界清楚之后，我们就可以进入搭建层了。

一个最小 Hermes Agent，不需要太多部件，但至少应该包括下面这些内容：

1. 角色定义

也就是这个 Agent 是干什么的。

例如：

- 你是一个内容整理助手
- 你的任务是从输入文本中提取关键信息
- 你必须输出结构化摘要
- 不得杜撰原文不存在的信息

角色定义不是为了“写得像 AI”，而是为了明确职责。职责越清晰，后面行为越稳定。

2. 输入定义

也就是这个 Agent 接收什么输入。

例如：

- 原始文本
- 可选附加要求
- 输出格式要求

输入定义的价值，在于避免模糊任务。Agent 很怕模糊输入，输入越含糊，输出越飘。

3. 输出定义

也就是结果必须长什么样。

比如可以要求输出为：

- 主题
- 核心观点
- 关键事实
- 结论

这一步看起来简单，实际极其重要。因为很多 Agent 失控，并不是它不会做，而是你没规定它“做成什么样才算完成”。

4. 工具定义

对于这个最小例子，工具可以很少，甚至只要一个读取文本输入的能力，加上一个输出写入能力就够了。

这里的原则是：

只给必要工具，不给多余工具。

第一次搭建时，工具少反而更好，因为每多一个工具，就多一层决策噪音。

5. 执行流程

即便是最小 Agent，也最好有个简单流程：

1. 接收输入
2. 分析文本
3. 提取重点
4. 按格式输出

这就是最基础的 Workflow。哪怕这个流程不显式写成图，只要你脑子里有这个顺序，系统就会稳定很多。

3.5 第一次运行时，你真正要看什么



当第一个 Hermes Agent 跑起来之后，很多人会本能地只看结果，觉得“输出还行”就结束了。其实这远远不够。

第一次运行时，你最应该看的，不只是最终结果，而是这几个层面：

1. 它有没有正确理解任务

比如你让它做结构化摘要，它到底是在总结原文，还是在自由发挥，还是在泛泛而谈。

如果目标理解一开始就歪了，后面再怎么调格式都没用。

2. 输出结构稳不稳

不是一次能对，而是多跑几次是否基本一致。

如果结构每次都漂，那通常说明行为边界不够清楚，或者输出约束太弱。

3. 有没有不必要的发挥

第一次搭建最常见的问题之一，就是 Agent 过度聪明。

明明让它整理内容，它却开始补背景、加解释、扩展观点，甚至替你做额外推理。这些能力不是绝对不好，但如果不受控，后面一定出事。

4. 工具调用是否合理

如果你的 Agent 已经接了工具，就要看它是否真的在该用的时候才用，而不是乱调、漏调或反复调用。

5. 出错时能不能看懂

第一次运行不怕错，怕的是错了你看不懂。

真正健康的最小系统，不是“永远不出错”，而是“出了错你知道怎么修”。

3.6 第一次搭建最常见的坑



到这里，本王先替你踩掉几个最常见的坑。

坑一，上来就想做全能 Agent

这是最常见也最致命的坑。第一次搭建时，目标一定要小，职责一定要单一。越想一步到位，越容易原地爆炸。

坑二，Prompt 写得很长，但规则不清楚

提示词长，不等于系统稳。真正关键的不是字数，而是目标、边界、输出、异常处理是否说清楚。

坑三，工具接太多

很多人喜欢把一堆工具先全接上，觉得以后总能用上。结果 Agent 决策空间过大，频繁误用工具，反而把系统搞乱。

坑四，只看结果，不看过程

只看输出长得像不像，忽略执行链路，是后面难以优化的根源。真正的调试不是“这次看着还行”，而是“我知道它为什么这样输出”。

坑五，一开始就做多智能体

多智能体不是新手村教程。第一次搭建，请先把单 Agent 最小闭环打通。否则你只是把一个问题变成三个问题，再用调度把它们缠到一起。

本章小结



这一章的核心任务，是帮你完成一个关键转变：

从“理解 Hermes Agent 是什么”，进入“开始动手搭建第一个 Hermes Agent”。

我们强调了一个非常重要的原则：

第一次搭建，不追求最强，只追求最小闭环。

只要这个闭环里包含了明确目标、清晰角色、必要工具、基础流程和可观察结果，你就已经跨过了最难的一步。后面无论接工具、加记忆、做评估、拆多智能体，都是在这个稳定起点上逐步展开。

如果没有这个起点，后面所有复杂能力都只是空中楼阁。

延伸阅读



进入下一章之前，建议继续带着这几个问题：

- 一个 Hermes Agent 的行为边界，应该写到多细
- Prompt、规则、Workflow 三者之间应该怎么分工
- 工具什么时候该加，什么时候不该加
- 当第一个 Agent 已经能跑时，下一步优先优化什么

第四章 提示词设计与行为控制



很多人第一次接触 Agent 系统时，会天然把注意力全压在提示词上，仿佛只要 prompt 写得足够长、足够细、足够“专业”，系统就会自动变稳。这个理解不算全错，但也只对了一半。

提示词当然重要，它确实会直接影响 Agent 的理解方式、表达风格、工具调用倾向和任务推进节奏。但真正决定系统是否稳定的，从来不只是提示词本身，而是提示词在整个系统里承担什么角色。

如果系统目标不清，流程混乱，工具边界模糊，输出约束缺失，那你就算写了一页漂亮 prompt，也只是给一个结构混乱的系统贴上了一层看起来很聪明的包装纸。短期也许能唬住几次演示，长期一定会失控。

所以这一章的目标，不是教你“怎么写一段厉害 prompt”，而是帮你建立一个更稳的认知：

提示词不是装饰文案，而是 Agent 行为控制系统的一部分。

当你这样理解它之后，后面写出来的东西才会真正有工程价值。

4.1 Prompt 的作用，到底是什么

先把这个问题说透。Prompt 的本质，不是让模型“文采更好”，而是让 Agent 在某个具体任务场景中，按照你希望的方式理解目标、约束行为并组织输出。

换句话说，prompt 干的是三件事：

- 帮 Agent 理解自己是谁
- 帮 Agent 理解当前要干什么
- 帮 Agent 理解什么能做，什么不能做

这三个层面，缺哪个都不行。

如果没有角色定义，Agent 就会飘。如果没有任务定义，Agent 就会泛。如果没有行为边界，Agent 就会乱。

所以，提示词最核心的价值，不是增强“表达”，而是增强“控制”。

这也是为什么很多人明明 prompt 写得很长，系统还是不稳。问题不在于不够长，而在于写的东西没有承担控制功能。比如有些 prompt 充满宏大口号，什么“你是世界级专家”“你拥有最强分析能力”“请尽可能全面深入地完成任务”，这些话听着挺猛，但对系统稳定帮助并不大。

真正有用的，不是形容词，而是约束。

例如：

- 你的职责是什么
- 输出必须采用什么结构
- 什么时候允许调用工具
- 遇到信息不足时怎么办
- 不得做哪些越界行为

这些东西，才是行为控制的骨架。

4.2 System Prompt，决定的是底层人格和边界

在一个 Hermes Agent 系统里，System Prompt 一般承担的是底层约束角色。它不应该被理解成“开场白”，而应该被理解成系统规则声明。

它解决的是下面这几个问题：

- 这个 Agent 的身份是什么
- 它承担什么职责
- 它遵循什么原则
- 它有哪些边界
- 它默认如何处理冲突、不确定性和异常

所以，System Prompt 更像宪法，不像临场任务单。

一个好的 System Prompt，通常不追求花哨，而追求稳定。它不应该每次任务都大改，更不应该把所有临时需求都往里塞。System 层一旦过于臃肿，后面你会很难判断问题到底出在系统角色、任务描述，还是流程设计。

通常来说，一个稳的 System Prompt，至少应该包含五部分：

1. 角色定义

先讲清楚这个 Agent 是什么。

比如：

- 你是一个内容整理助手
- 你是一个研究型分析 Agent

- 你是一个流程执行协调者

角色定义要具体，不要泛。“你是一个强大的 AI 助手”这种说法，基本没什么控制价值。

2. 核心职责

告诉它主要负责什么，不负责什么。

比如：

- 负责根据输入信息整理结构化输出
- 不负责凭空补充事实
- 不主动扩展无关内容

职责越清晰，Agent 越不容易乱跑。

3. 行为原则

这是 System Prompt 里非常关键的一层。也就是 Agent 在执行时应遵守的共通规则。

例如：

- 优先准确，再追求完整
- 信息不足时明确说明
- 不确定内容不作肯定判断
- 优先使用给定工具，不自行虚构外部信息
- 结果以可验证、可执行为导向

这些原则，决定了系统在模糊和复杂场景下的默认反应方式。

4. 输出约束

很多人把输出格式只写在用户提示词里，但如果某种结构是长期稳定要求，System 层也可以做底层约束。

比如：

- 输出尽量结构化
- 优先使用分点表达
- 若任务包含结论，需单独列出
- 不输出与任务无关的寒暄内容

这能显著减少系统输出风格漂移。

5. 工具与边界说明

如果是 Agent 系统，这一部分尤其重要。

比如：

- 只有在需要外部信息时才调用搜索工具
- 若工具失败，应说明失败而不是伪造结果
- 未经允许不得执行破坏性操作
- 若现有信息足以完成任务，不应多余调用工具

这一层，直接决定 Agent 会不会“乱动手”。

4.3 用户提示词，不是重复系统规则



很多人会犯一个很典型的错误，就是把 System Prompt 和用户提示词写成同一份东西，前面说一遍，后面再说一遍。这样做不仅浪费上下文，还会把控制逻辑搞得层级混乱。

更合理的分工应该是：

- System Prompt 负责长期稳定规则
- User Prompt 负责当前任务要求
- Workflow/代码逻辑 负责流程控制

也就是说，用户提示词不应该重复声明“你是谁”，而应该聚焦这次任务本身。

例如，System Prompt 已经定义了你是“结构化研究助手”，那用户层就没必要再写一遍“你是一个资深研究顾问”。用户层真正应该说的是：

- 本次任务是什么
- 输入材料是什么
- 这次希望输出什么
- 有哪些额外要求
- 本次判断标准是什么

这样分层后，系统会稳很多。否则所有规则都堆在一个地方，一旦结果不对，你根本不知道该调哪一层。

4.4 行为控制，真正靠的是“约束清晰”，不是“语气严厉”



这点本王得直接说。很多人写 prompt 时特别喜欢用很重的语气，比如：

- 必须!
- 严禁!
- 绝对不能!
- 务必严格遵守!

这种写法不是完全没用，但它的效果远不如大家想象得大。因为模型不是被语气吓住才守规则，而是因为规则本身足够清晰、足够具体、足够一致，才更容易稳定执行。

真正有效的控制，来自三点：

第一，规则具体

不要说“请尽量专业地回答”，要说“输出包括问题定义、核心分析、结论三部分”。

不要说“请避免幻觉”，要说“未在输入中出现的信息不得作为事实陈述”。

具体规则比宏观口号有用得多。

第二，规则可操作

规则不能只停留在价值判断层，还得让系统知道该怎么执行。

比如“保持严谨”是价值判断，而“若无法确认事实，请明确标注‘待确认’”就是可执行规则。

第三，规则不要互相打架

这是很多 prompt 失控的根本原因之一。

你一边要求“尽量全面”，一边要求“尽量简洁”；一边要求“大胆推理”，一边要求“不得超出原文”；一边要求“主动调用工具”，一边又要求“减少调用成本”。

这些要求不是不能共存，但你必须给出优先级。否则系统只能随机取舍。

所以真正成熟的提示词设计，不只是堆要求，而是安排要求之间的层级关系。

4.5 如何让 Agent 少跑偏



Agent 跑偏，通常不是突然疯了，而是因为前面某一层设计含糊了。常见的跑偏有三种：

- 目标跑偏
- 输出跑偏
- 行为跑偏

目标跑偏

比如你本来要它整理内容，它却开始做观点发挥。这种问题一般来自任务定义太模糊。

解决方法不是加一句“别跑偏”，而是把目标写得更明确：

- 任务是提取，不是评论
- 任务是整理，不是扩写
- 任务是总结，不是改写风格

输出跑偏

比如你要结构化结果，它却写成大段散文。这通常说明输出约束太弱。

解决办法是直接给结构骨架，而不是只说“请结构化输出”。

行为跑偏

比如本来不需要查外部资料，它却频繁调用工具；或者明明缺信息，仍然硬编结果。

这种问题一般出在工具规则与异常处理规则没写清。

你要明确告诉它：

- 什么时候该停
- 什么时候该问
- 什么时候该说明不足
- 什么时候该调用外部资源

Agent 真正稳，不是因为它什么都会，而是因为它知道什么时候不乱来。

4.6 Prompt 不是万能的，别拿它替代系统设计



这一节必须讲，因为太多人会掉进这个坑。

当 Agent 输出不好时，很多人的第一反应永远是改 prompt。这并不是完全错，但很多问题其实根本不是 prompt 问题。

比如：

- 工具定义不清
- Workflow 路径不合理
- 输出校验缺失
- 任务拆分本身就不对
- 输入信息质量太差

这些问题，你再怎么改措辞，也救不回来。

所以，成熟的做法应该是把问题分层看：

- 是 System 角色层的问题
- 是任务描述层的问题
- 是工具调用层的问题
- 是流程设计层的问题
- 还是评估与反馈层的问题

只有分层，你才调得动。否则所有问题都丢给 prompt，本质上就是拿文案补架构漏洞。

本章小结



这一章最核心的结论可以压成一句话：

提示词的本质，不是包装语言，而是行为控制。

一个 Hermes Agent 能不能稳定执行任务，很大程度上取决于你是否把角色、职责、边界、输出要求和工具规则讲清楚了。真正有效的 prompt，从来不是写得最像“专家”的那种，而是写得最清楚、最一致、最可执行的那种。

同时我们也强调，prompt 很重要，但它不是万能药。当系统开始复杂起来时，行为控制一定是 System Prompt、任务描述、Workflow、工具约束和结果校验共同作用的结果，而不是靠一段神奇文案单点拯救。

延伸阅读



带着这几个问题进入下一章会很有帮助：

- 工具到底应该什么时候接入
- 工具越多是否越强
- 工具调用失败后，系统该如何处理
- 真正可落地的 Agent 工具链，应该怎么设计边界

第五章 工具接入，让 Agent 真正干活



如果说大模型让 AI 学会了“说话”，那么工具接入，才真正让 Agent 学会“做事”。

这是 Agent 系统和普通聊天系统之间最本质的分水岭之一。没有工具的 Agent，再聪明，本质上也还是一个困在上下文里的语言处理器。它能分析、能解释、能总结、能建议，但它无法真正触碰外部世界。不能读文件，不能调接口，不能查网页，不能写结果，不能和其他系统发生真实交互。

而一旦接入工具，情况就完全不一样了。

Agent 不再只是回答问题，而开始具备行动能力。它可以读取本地资料、访问知识库、抓取网页信息、调用业务 API、写入数据库、生成文件、触发自动化流程，甚至向其他 Agent 或系统发起协作请求。到了这一步，Agent 才真正开始从“智能问答”走向“数字执行体”。

所以，本章的重点不是“怎么给 Agent 多装几个工具”，而是先搞清楚一个更关键的问题：

为什么工具接入，会让 Agent 发生质变。

5.1 没有工具的 Agent，为什么总差一口气

很多人第一次做 Agent 时，会先在纯对话环境里试几轮，觉得已经挺聪明了。能写总结、能拆任务、能给方案，甚至还能假装像模像样地做分析。可一旦把它放进真实业务场景，这种“看起来挺行”的状态很快就会露馅。

原因很简单，因为真正的业务任务很少只靠语言就能完成。

比如：

- 你让它“帮我总结这个项目文件夹的结构”，它必须真的去读文件
- 你让它“查一下这个产品的最新信息”，它必须真的去搜索
- 你让它“把结果保存成报告”，它必须真的去写文件
- 你让它“把这条消息同步给团队”，它必须真的去调用消息接口

如果这些动作都做不到，那它就只能停留在“我建议你这么做”的层面。说白了，就是只会当参谋，不会下场干活。

这就是为什么很多 Demo 看起来惊艳，但一进真实环境就显得虚。它不是不会推理，而是没有行动抓手。

所以，工具接入并不是锦上添花，而是 Agent 落地的基础设施。没有工具，Agent 只是能描述世界。有了工具，Agent 才开始能够影响世界。

5.2 Tool 的本质，不是插件，而是能力边界

讲工具时，很多人习惯把它理解成“给 Agent 加几个插件”。这种说法不算错，但太轻了，会让人低估它的系统意义。

更准确地说，Tool 是 Agent 获得外部行动能力的接口。它决定的，不只是“能不能调用某个功能”，而是“Agent 的行为边界到底延伸到哪里”。

比如一个只有文本总结能力的 Agent，它的边界就在上下文之内。如果你给它加上本地文件读取工具，它的边界就延伸到了文件系统。如果再给它加网页搜索工具，它的边界又延伸到了外部信息空间。如果再接入数据库或业务 API，它就开始触碰真实业务系统了。

所以 Tool 的本质，不只是“能做更多事”，更是“决定这个 Agent 实际拥有多大行动半径”。

这就带来一个非常关键的设计原则：

工具不是越多越好，边界也不是越大越好。

因为 Agent 一旦拥有某种能力，就意味着系统必须为这种能力承担对应的成本和风险。能力越强，系统越要回答这些问题：

- 什么时候允许它调用
- 调用错了怎么办
- 调用失败怎么处理
- 调用结果如何验证
- 哪些操作绝对不能让它碰

也就是说，每加一个工具，实际上都是在给系统增加一条新的责任链。

5.3 常见工具类型，可以怎么分类



在真实项目里，Agent 工具通常会落在下面几类。分类的意义，不是为了学术整齐，而是为了帮你更快判断一个任务到底缺什么能力。

第一类，信息获取类工具

这类工具负责帮 Agent 获取外部信息。最常见的包括：

- 网页搜索
- 页面抓取
- 文档读取
- 知识库检索
- 数据库查询

这类工具的作用，是让 Agent 不再只依赖输入上下文，而是能够主动补充完成任务所需的信息。

比如研究类 Agent、内容分析类 Agent、情报整理类 Agent，往往都严重依赖信息获取类工具。

第二类，内容处理类工具

这类工具负责把已有信息做进一步转换、处理或生成。

例如：

- 文本清洗
- 文件格式转换
- Markdown / PDF / 表格生成

- 数据结构整理
- 图表生成

这类工具的意义，在于帮助 Agent 把“理解过的信息”变成“可交付的结果”。

第三类，执行操作类工具

这类工具会真正触发动作。比如：

- 写文件
- 发消息
- 调用业务接口
- 创建任务
- 更新数据库
- 执行脚本

到了这一层，Agent 已经不是只在分析内容，而是在直接参与业务动作了。所以这一类工具的风险和价值通常都更高。

第四类，协作调度类工具

当系统开始有多个 Agent，或者需要和其他系统协同时，就会出现这类工具。

例如：

- 调用其他 Agent
- 分发任务
- 读取共享状态
- 汇总其他角色结果
- 触发 workflow 节点

这类工具的意义，是把一个单点 Agent 扩展成可协作系统。

5.4 第一次接工具，为什么一定要克制



本王得直接说，很多 Agent 项目一烂，就是从“工具接太多”开始烂的。

刚接触 Agent 的人很容易兴奋，觉得工具越多，系统越厉害。于是网页搜索接上，数据库接上，文档读取接上，发送消息接上，自动执行脚本也接上。看起来像是给 Agent 装了满身神兵利器，实际上很可能是给自己埋了一地雷。

因为每多一个工具，系统就会多出几层复杂度：

- Agent 要不要选它
- 什么时候选它
- 会不会误选
- 调用结果怎么处理
- 失败后怎么回滚

如果这些问题没设计清楚，工具数量越多，系统越混乱。到最后你会看到一种很典型的场景：

- 本来不需要搜索，它搜索了
- 本来该直接输出，它先乱调工具
- 本来该停下来报错，它硬编结果
- 本来一个动作就够，它反复调用三四遍

这不是模型坏了，而是你给了它太多选择，却没给足够清晰的决策规则。

所以第一次接工具时，最好的原则不是“尽量全”，而是：

只接完成当前任务真正必要的那一两个工具。

比如一个最小内容整理型 Agent，可能只需要：

- 一个读取输入内容的能力
- 一个输出结果的能力

够了，先别贪。

5.5 工具调用什么时候该发生



这是一个特别关键、也特别容易被忽视的问题。

很多人接完工具之后，只在意“能不能调”，却不太关心“什么时候该调”。但对 Agent 系统来说，后者往往比前者更重要。

因为工具调用不是展示动作，而是任务决策的一部分。如果时机不对，调用越成功，结果反而越乱。

一般来说，一个成熟的 Agent 至少要回答清楚下面几种情况：

1. 现有信息已经够了，还要不要调工具

通常不该调。如果输入里信息已经足够完成任务，再调工具往往只是增加延迟、成本和噪音。

2. 现有信息不够，但可以通过工具补足，要不要调

这时应该调。但前提是这个缺口是客观缺口，而不是 Agent 自己没先把已有信息用好。

3. 工具可能有帮助，但不是必须，要不要调

这时要看任务目标。如果任务追求速度和成本控制，可以不调。如果任务追求准确性和实时性，可能值得调。

4. 工具调用失败了怎么办

这是很多系统最容易露馅的地方。真正成熟的 Agent，不是工具失败后还假装没事，而是会明确报告失败，并根据规则决定是否重试、降级或终止。

所以，工具调用不是“有了就用”，而是“在合适的地方、为合适的目的、按合适的规则使用”。

5.6 工具失败，不要让 Agent 硬装成功



这一节必须单独拎出来讲。

很多 Agent 系统最危险的地方，不是工具用得少，而是工具失败后 Agent 还装作自己完成了任务。比如网页没抓到，它却根据记忆胡乱补。接口报错了，它还继续生成一个看起来像成功结果的输出。文件没写进去，它却告诉你“已保存”。

这种行为在 Demo 里可能不显眼，但一旦进业务，杀伤力极大。因为这会直接破坏系统可信度。用户最怕的不是 Agent 会失败，而是它失败了还嘴硬。

所以在 Hermes Agent 系统里，工具失败后的处理规则必须尽量清晰，至少要覆盖这几种动作：

- 明确报错
- 说明失败位置
- 说明失败原因
- 必要时重试
- 若无法恢复，返回降级结果或中止任务

你会发现，这其实已经不只是工具设计问题，而是系统可信性设计问题。Agent 真正可靠，不是从不犯错，而是犯错时不骗人。

5.7 从“能调用工具”到“工具链可落地”



当我们说“Agent 接了工具”，很多人脑子里浮现的是功能越来越多。但对于真正落地的系统来说，更重要的问题其实是：

这些工具能不能组成一条稳定的任务链。

比如一个简单的业务流程可能是：

1. 搜索信息
2. 抓取网页
3. 提取结构化内容
4. 生成报告
5. 保存结果
6. 发送通知

单看每一步，都是一个独立工具调用。但系统真正的价值，不在于这六个动作都能分别跑通，而在于它们能不能稳定串起来。

这就要求你在设计工具链时，开始思考更系统的问题：

- 前一步的输出能否直接成为下一步输入
- 数据格式是否统一
- 错误会在哪一段中断
- 中断后是否能恢复
- 哪一步最该做校验
- 哪一步最该人工兜底

到了这一步，你就已经从“工具接入”走向“工具链设计”了。而这，才是 Agent 从玩具变成系统的关键转折点。

本章小结



这一章最核心的结论可以压成一句话：

工具接入，决定 Agent 能不能真正干活。

没有工具，Agent 只能理解和表达。有了工具，Agent 才开始具备与外部世界互动的能力。

但我们也反复强调，工具不是越多越强。真正成熟的做法，是围绕具体任务选择必要工具，清楚定义调用时机、失败处理、权限边界和链路衔接。否则工具越多，系统越容易变成一个看起来很能干、实际上很不稳的拼装怪。

从某种意义上说，Agent 工程的真正难点，并不只是把工具“接上”，而是让工具能力被系统稳定地“用起来”。

延伸阅读



带着这几个问题进入下一章会很有帮助：

- 任务一旦变成长流程，信息该如何持续保存
- 上下文窗口和长期记忆到底该怎么分工
- 什么信息应该记住，什么信息不该记住
- 为什么很多 Agent 用久了会越来越乱

第六章 记忆与上下文管理



如果说工具接入解决的是 Agent “怎么行动”的问题，那么记忆与上下文管理，解决的就是 Agent “如何持续保持判断一致性”的问题。

很多人第一次做 Agent 时，最容易产生一种错觉，觉得只要模型上下文窗口够大，记忆问题就自然解决了。能塞进去的内容足够多，不就等于 Agent 有记忆了吗？

听起来好像有点道理，实际上差得很远。

上下文窗口和记忆，从来不是一回事。上下文解决的是：当前这一轮，模型眼前能看到什么。记忆解决的是：哪些信息应该被持续保留，何时被取回，以及取回后如何影响接下来的决策。

这是两个完全不同层级的问题。

如果你搞不清这个区别，系统一开始可能还能跑，跑久了就很容易变得越来越乱。旧信息堆积、任务状态污染、历史偏好误用、上下文冗余膨胀、关键约束丢失，这些问题最后都会一起涌上来。很多人以为是模型不稳，实际上是记忆设计从一开始就没立住。

所以这一章，我们要把一个非常关键的认知钉牢：

Agent 不是“记得越多越好”，而是“记得对、取得准、忘得掉”。

6.1 上下文窗口，不等于记忆系统



先把这个误区狠狠干碎。

上下文窗口本质上是一块临时工作台。你这次把什么放上去，模型这轮就能看到什么。可一旦任务结束，或者下一轮输入替换了内容，这块工作台上的东西就可能被挤掉、覆盖，甚至彻底丢失。

所以，上下文窗口更像短时注意力，不像真正的记忆。

它的作用很重要，但它不负责长期保存，也不负责主动检索，更不负责信息分层。你不能因为桌子够大，就说自己家有图书馆。一个是眼前能摊开的资料，一个是知道什么该存、存在哪、什么时候取出来用。

在 Agent 系统里，如果你把所有“该记住的东西”都直接堆进上下文，通常会出现三类问题：

第一，越堆越乱

任务一多、轮次一长，系统就开始拖着越来越长的历史信息走。看起来像是“上下文更完整了”，实际上噪音也同步增加了。很多无关细节会继续挂在上下文里，反而干扰当前判断。

第二，关键信息反而容易淹没

上下文越长，不代表重要信息越突出。恰恰相反，如果没有设计层次，真正关键的规则、偏好和任务状态，很容易被埋在大量历史文本里。模型虽然“看到了”，但不代表它能稳定抓住重点。

第三，成本和延迟一起上升

上下文不是免费的。塞得越多，调用成本越高，响应越慢，而且一致性不一定更好。很多系统到后面开始越来越笨重，本质上就是因为把上下文当垃圾仓库用了。

所以，上下文一定要用，但它必须被看作当前任务的工作内存，而不是记忆本体。

6.2 记忆的本质，是“任务连续性基础设施”



那什么才叫记忆？

在 Agent 系统里，记忆的本质不是存数据本身，而是为连续任务提供稳定的信息延续能力。它解决的是：上一次发生的事情，下一次是否还应该影响判断。更进一步，它还解决：哪些历史信息需要被留住，哪些必须丢掉，哪些只在特定条件下才应该重新进入上下文。

从这个角度看，记忆不是“存储附件”，而是任务连续性的基础设施。

一个没有记忆设计的 Agent，每一轮都像临时工。你让它做一件事，它能做；下一件事来了，它又像第一次见你。即便它在当前轮次看起来很聪明，也很难在跨轮、多步骤、长期任务里保持真正稳定。

而一旦有了合适的记忆机制，系统就开始表现出一种更接近“持续合作对象”的特征。它知道你当前在推进什么任务，知道之前定过什么规则，知道哪些信息已经确认过，知道哪些历史判断应该继续沿用。

所以，记忆真正的价值不是“让 Agent 显得聪明”，而是“让系统在时间维度上保持连续性”。

6.3 短期记忆，服务当前任务过程



记忆一般至少要分两层来理解。第一层，就是短期记忆。

短期记忆服务的是当前任务过程。它主要保存那些在本轮或当前阶段执行中会反复用到、但未必必要长期保留的信息。

例如：

- 当前任务目标
- 中间推理结果
- 已完成的步骤
- 当前子任务状态
- 上下文中已确认的约束
- 这轮刚得到的工具返回结果

这些信息的特点是，它们对“眼下这件事能不能顺利推进”非常关键，但未必值得永久留存。

你可以把短期记忆理解成任务现场白板。写在上面的东西，是为了这次协作过程顺畅。任务结束后，白板上的很多内容应该被擦掉，只保留最终有价值的结论。

如果短期记忆做不好，会有两种典型问题：

一种是“断片”

Agent 刚刚做过的步骤，下一轮就忘了，导致重复劳动、状态错位、流程跳步。

另一种是“拖泥带水”

本该随着任务结束而消失的过程信息，被一直拖进后续任务，结果系统开始带着一堆过期状态继续跑，越跑越浑。

所以，短期记忆最重要的不是“留住更多”，而是“让当前任务够连贯，同时及时清理”。

6.4 长期记忆，服务跨任务复用



和短期记忆不同，长期记忆服务的是跨任务、跨轮次、跨时间段的持续复用。

它保存的通常不是某一次具体执行过程中的每个细节，而是那些值得长期影响系统决策的信息。

例如：

- 用户的长期偏好
- 稳定的规则与约束
- 重要的事实资料
- 已验证过的经验
- 项目背景
- 固定术语和表达标准
- 历史决策及其原因

长期记忆更像一个经过筛选的知识层，而不是流水账。它最忌讳的不是记少了，而是把太多未经提炼的原始过程都永久挂进去。

因为一旦长期记忆变成杂物间，系统很快就会开始误取、乱取、过度依赖旧信息。最后的结果不是“更有连续性”，而是“被历史拖死”。

所以长期记忆设计的关键，不只是存，而是筛。要问的不只是“这条信息以后会不会有用”，而是：

- 它是不是足够稳定
- 它是不是会多次复用
- 它是不是值得长期影响系统行为
- 它以后被重新取出时，会不会比有帮助更容易造成干扰

只有通过这层筛选，长期记忆才会成为系统资产，而不是系统负担。

6.5 不是所有信息都该记住



这是本章最重要的一条铁律之一。

很多人做 Agent 记忆时，天然会有一种仓鼠心态，觉得既然以后可能用得上，那就先全存着。反正存下来总比丢了强。听起来很安全，实际上很危险。

因为在 Agent 系统里，错误的记忆比没有记忆更糟。

没有记忆，最多是重新问一次、重新做一次。错误记忆一旦被重新取用，系统会带着错误的“自信”继续执行，这比单纯不知道更致命。

所以，不是所有信息都值得进记忆。通常来说，下面这几类内容要格外谨慎：

1. 临时过程噪音

比如中间试错记录、某一步未确认的推测、阶段性临时安排，这些内容往往只对当前过程有效，不适合直接进入长期记忆。

2. 低置信度结论

如果某条信息本身就没有核实过，只是 Agent 根据上下文推出来的临时猜测，那它绝对不该直接作为稳定记忆存下来。

3. 高频变化信息

比如今天有效、明天可能失效的临时状态，除非系统本身支持版本和过期机制，否则很容易污染后续判断。

4. 过于细碎的原始日志

完整日志当然有其价值，但它更适合做归档，不适合直接作为高频调用记忆源。记忆层最好保存提炼过后的结构化结论，而不是无限增长的流水文本。

记忆设计里，真正重要的能力，其实不是“如何记”，而是“如何不乱记”。

6.6 好的记忆系统，一定要支持检索和回收

很多人设计记忆，只想到“存进去”，却没认真想过“怎么取出来”。这是典型的半套系统思维。

如果一条信息被存了，但后面很难被正确检索到，那它就只是躺在仓库里的死数据。反过来，如果系统会频繁把不该拿出来的旧信息拿出来，那这个记忆系统也一样有毒。

所以一个靠谱的 Agent 记忆系统，至少要同时具备两种能力：

第一，能正确检索

也就是在合适的时候，把真正相关的信息拿出来，而不是一股脑全倒进上下文里。

这意味着记忆不应该只是文本堆，而应该有某种检索逻辑，比如：

- 按任务主题取
- 按项目取
- 按用户偏好取
- 按时间取
- 按标签取
- 按相似语义取

只有这样，记忆才能真正参与当前任务，而不是成为历史废墟。

第二，能及时回收

回收能力经常被忽视，但实际上特别关键。因为没有回收机制的记忆系统，最后几乎都会膨胀成信息垃圾场。

回收不一定意味着真的删除，也可以是：

- 降低优先级
- 标记为过期
- 转入归档层
- 不再默认参与检索
- 只在特殊条件下再取出

你会发现，这和人类记忆其实很像。真正高效的记忆，不是永远紧抱所有细节不撒手，而是知道什么该长期活跃，什么该沉下去。

6.7 记忆污染，是 Agent 系统最隐蔽的慢性病

前面说了这么多，本王最后得把一个最阴的坑单独拎出来：记忆污染。

什么叫记忆污染？就是系统把本不该长期影响决策的信息，当成了稳定依据，结果后续任务开始持续受到误导。

它通常不会像工具报错那样一下炸给你看。它更像慢性病，一开始不明显，但越用越别扭，最后你会感觉系统“怎么越来越不对劲了”。

常见污染方式有几种：

- 旧任务状态混入新任务
- 临时要求被错当长期偏好
- 错误事实进入长期记忆
- 已过期规则继续生效
- 某次失败经验被过度泛化

记忆污染最可怕的地方在于，它很容易伪装成“系统有连续性”。表面上看，Agent 好像记住了很多东西，实际上它记住的是一堆过期、错误或不该复用的信息。

所以，任何想做长期可用 Agent 的系统，都必须把“防污染”当成基本工程要求，而不是后期再补的优化项。

本章小结



这一章最核心的结论，可以压成一句话：

记忆不是为了存更多，而是为了让系统在时间维度上持续稳定。

我们区分了上下文窗口和真正的记忆系统，也区分了短期记忆与长期记忆的不同职责。短期记忆服务当前任务连贯性，长期记忆服务跨任务复用，而真正成熟的系统，不只会记，还必须会筛、会取、会忘、会回收。

如果工具接入让 Agent 开始真正“行动”，那么记忆与上下文管理，决定的就是它能不能在连续行动中保持稳定、不失真、不自我污染。

延伸阅读



带着下面几个问题进入下一章会很有帮助：

- 当任务复杂到一个 Agent 很难稳住时，什么时候该拆成多 Agent
- 多个 Agent 的分工边界应该怎么定
- 串行、并行、路由这几种协作模式该怎么选
- 多 Agent 协作时，状态和记忆又该如何传递

第七章 多智能体协作系统设计



当一个 Agent 系统开始变复杂时，迟早会碰到一个问题：单个 Agent 还能不能继续扛下去。

一开始，很多任务用单智能体就够了。目标清楚、流程短、工具有限、上下文不算太复杂，一个设计得好的单 Agent 不仅成本低、好调试，而且往往比花里胡哨的多智能体系统更稳。

但任务一旦变长、变深、变杂，问题就会开始冒出来。一个 Agent 既要理解需求，又要搜集资料，又要做规划，还要执行细节、检查结果、处理异常。这时候它不是不能做，而是开始越来越容易出现认知负担过重、状态混乱、角色漂移和决策失真。

很多人第一次意识到需要多智能体，往往就是在这个节点。不是因为“多 Agent 很先进”，而是因为单 Agent 已经快被塞爆了。

所以，多智能体协作系统真正要解决的问题，不是“怎么让几个 Agent 互相聊天”，而是：如何通过合理分工，把一个复杂任务拆成多个可稳定执行的部分。

7.1 为什么需要多智能体



多智能体系统最大的价值，不在于它比单智能体更炫，而在于它能够通过分工降低复杂任务的系统负担。

你可以把它理解成一个现实团队。

如果一个人同时负责调研、写作、审稿、排版、发布、复盘，他可能也不是完全做不了，但效率、稳定性和质量都很难长期维持。一旦拆成几个角色，每个人负责一段边界清晰的工作，整体系统反而更容易稳下来。

在 Agent 系统里也是一样。

一个复杂任务通常包含几种不同类型的工作：

- 信息搜集
- 资料判断
- 任务规划
- 内容生成
- 质量审查
- 工具执行
- 结果汇总

这些工作对能力侧重点的要求并不相同。有的偏探索，有的偏结构化，有的偏审慎，有的偏执行。如果把它们全塞给一个 Agent，它迟早会开始在角色之间来回打架。

所以，多智能体的核心逻辑其实很朴素：

不是让每个 Agent 更强，而是让每个 Agent 更专。

7.2 什么时候该拆，不要太早，也不要太晚



多智能体当然有价值，但也不是越早拆越好。拆早了，你会得到一个复杂度明显高于任务本身的系统。拆晚了，单 Agent 已经混成一锅粥，问题很难再局部收敛。

通常来说，当出现下面这些信号时，就说明该认真考虑拆分了：

第一，任务类型明显异构

比如既有检索判断，又有内容创作，又有执行操作。这些环节的思维模式差别很大，让一个 Agent 全包，很容易顾此失彼。

第二，上下文负担越来越重

一个 Agent 必须同时背太多状态，导致判断不稳定、信息抓取失焦，说明任务已经超出单体负荷了。

第三，任务链条开始变长

如果一个任务天然有多个阶段，而且每阶段都有明确产物，那就非常适合拆成不同角色分步处理。

第四，质量控制要求提高

当你不再满足于“能跑出来”，而是要求结果可审、可控、可复查时，多角色分工会明显有利于质量治理。

第五，扩展与维护开始吃力

如果系统每加一个能力，都要把原有 Agent prompt 和流程大改一遍，说明单体系统已经接近维护边界了。

所以，多智能体不是起点，而是当复杂度逼近单体极限时的一种系统升级手段。

7.3 多智能体系统里，最常见的几种角色



虽然每个业务系统的角色设计都不同，但在大多数场景里，多智能体系统通常会出现几类高频角色。

主控 Agent

主控 Agent 负责理解总目标、拆分任务、调度流程、汇总结果。它不一定亲自做所有细活，但它要知道系统现在在做什么、下一步该交给谁，以及最终结果要如何收口。

你可以把它理解成项目经理，不是产出所有成果的人，但它控制整体推进节奏。

研究 Agent

研究 Agent 负责搜集信息、提取资料、补充上下文、汇整外部输入。它的重点不是表达，而是信息质量。

规划 Agent

规划 Agent 负责把模糊目标拆成可执行步骤，明确顺序、依赖和产出。它的价值不在于干活快，而在于把任务拆得清楚。

执行 Agent

执行 Agent 负责真正把动作做下去，比如调用工具、写文件、运行脚本、处理具体业务步骤。

审查 Agent

审查 Agent 负责检查一致性、完整性、格式正确性、逻辑漏洞和质量缺口。它不追求花活，它追求的是“别让脏结果混出去”。

在实际系统里，这些角色不一定都要独立存在。有时研究和规划可以合并，有时执行和审查可以部分合并。关键不是角色名字，而是职责边界清不清楚。

7.4

串行、并行、路由，这三种协作模式有什么区别



多智能体系统真正开始有意思，也真正开始变难，是从协作模式开始的。

最常见的协作模式，基本可以归为三种：

串行协作

串行最简单，也最稳。前一个 Agent 的结果，作为后一个 Agent 的输入，按顺序往下走。

例如：

1. 研究 Agent 搜集资料
2. 规划 Agent 生成大纲
3. 写作 Agent 输出内容
4. 审查 Agent 检查质量

串行的优点是流程清楚、责任明确、问题容易定位。缺点是速度偏慢，前一步出问题会一路传染下去。

并行协作

并行适合多个子任务彼此相对独立，可以同时推进。

例如一个研究任务，可以让多个 Agent 同时去：

- 查背景资料

- 查竞品
- 查用户反馈
- 查技术资料

最后再由汇总 Agent 统一收口。

并行的优点是快，特别适合信息搜集、批量处理和多路探索。缺点是结果整合难度更高，而且不同 Agent 输出风格和深度可能不一致。

路由协作

路由模式更像一个分发器。系统先判断当前任务属于哪一类，再把任务送给合适的 Agent。

例如：

- 技术问题送给技术 Agent
- 内容任务送给写作 Agent
- 数据分析送给分析 Agent
- 调用外部系统交给执行 Agent

路由的优点是资源利用合理，结构清晰。缺点是对分类逻辑要求高，一旦路由判断错了，后面全跑偏。

实际系统里，很多成熟架构不是只用一种模式，而是这三种混合使用。例如先路由，再局部并行，最后串行汇总。

7.5 多智能体真正难的，不是拆，而是协同



很多人以为多智能体系统最难的是“想出几个角色”，其实远远不是。真正难的是这些角色拆完之后，怎么让它们不互相添乱。

因为一旦系统变成多个 Agent，新的问题会立刻出现：

- 信息在谁手里
- 哪些状态要共享，哪些不该共享
- 一个 Agent 的错误会不会污染别的 Agent
- 输出格式不一致怎么办
- 多个 Agent 结论冲突怎么办
- 谁来拍板

这时候你会发现，多智能体系统本质上不只是“多几个执行体”，而是开始进入真正的系统工程状态。角色之间的接口定义、输入输出协议、共享上下文边界、异常处理规则，都会变得非常重要。

所以，多智能体系统要稳，至少要把三个东西想清楚：

1. 交接格式

前一个 Agent 交给后一个 Agent 的内容，最好尽量结构化，而不是一大坨自然语言。

2. 状态边界

哪些信息是公共状态，哪些只属于某个角色，不要糊成一锅。

3. 仲裁机制

如果多个 Agent 输出冲突，系统总得有最后拍板的人或规则。否则协作最后会变成互相打架。

7.6 多智能体系统最容易翻车的几个坑



坑一，为了显得高级而硬拆

任务明明一个 Agent 就够，非要拆成五个，最后复杂度远高于收益。这种系统通常看着热闹，实际维护成本爆炸。

坑二，角色定义过虚

什么“战略 Agent”“智慧 Agent”“全局洞察 Agent”，名字一个比一个响，实际职责模糊得一塌糊涂。角色设计最怕虚，必须能落到“它到底负责哪一步”。

坑三，所有 Agent 都能看所有信息

听起来像信息共享，实际上特别容易污染和失焦。不是所有 Agent 都需要看到全部上下文，很多时候恰恰要控制它们“只看自己该看的”。

坑四，没有统一收口层

多个 Agent 输出了一堆内容，却没有一个明确的汇总和裁决机制，最后系统变成信息堆，而不是结果系统。

坑五，把协作当对话表演

让多个 Agent 互相聊来聊去，表面上很像在协作，实际上没有明确目标推进，也没有稳定接口。这种系统演示时可能挺唬人，真落地通常一地鸡毛。

本章小结



这一章最核心的结论可以压成一句话：

多智能体不是为了炫技，而是为了通过分工让复杂任务更稳。

我们讨论了为什么需要多智能体、什么时候该拆、常见角色有哪些，以及串行、并行、路由三类典型协作模式。更重要的是，我们强调了一个常被忽视的事实：多智能体系统真正难的，从来不是“多几个 Agent”，而是“让多个 Agent 之间的协作接口足够清楚”。

如果第五章解决的是 Agent 如何行动，第六章解决的是信息如何延续，那么这一章解决的，就是复杂任务如何通过分工变得可控。

延伸阅读



接下来进入实战前，建议继续带着下面这些问题：

- 如果真要做完整业务系统，需求应该怎么拆
- 多智能体结构怎么映射到真实业务场景
- 从 0 到 1 做一个系统时，先搭哪部分最稳
- 如何在真实案例里把前面讲过的 Tool、Memory、Workflow、Multi-Agent 串成闭环

第八章 实战案例，从 0 到 1 搭一个业务系统



前面几章，我们一直在拆概念、讲原则、讲结构。到这一章，终于该把这些东西都拧到一块了。因为判断一本技术书到底有没有真东西，最直接的方式从来不是看前面概念讲得多漂亮，而是看它能不能把这些概念真正推到一个完整案例里，跑出闭环。

所以这一章，我们不再单讲某一个部件，而是要模拟一次真实项目落地过程。从需求出现开始，到任务拆解、架构设计、工具接入、记忆设计、执行流程，再到调试和复盘，完整走一遍。

这样做的意义很大。因为很多人学 Agent，一章一章看都懂，可一旦真让他自己起一个系统，就会突然发现脑子里是散的。知道 Tool 是什么，知道 Memory 是什么，知道多智能体怎么拆，但这些认知并没有真正变成系统搭建能力。

这一章的目标，就是补上这道坎：

让你看到一个 Hermes Agent 系统，如何从“想法”走到“可运行原型”。

8.1 先选一个真实、完整、边界清晰的业务场景



做案例，最怕两件事：

- 场景太空
- 场景太大

太空的例子看不出落地细节。太大的例子一写就假，最后只能停在宏观架构图和漂亮空话里。

所以这里，我们选一个非常适合用 Hermes Agent 落地，同时又足够完整的案例：

做一个“多智能体内容研究与成稿系统”

这个系统的目标是：

用户输入一个主题后，系统能够自动完成：

1. 主题理解
2. 资料搜集
3. 要点整理
4. 内容大纲生成
5. 初稿写作
6. 审校与结构优化
7. 输出最终可交付稿件

你会发现，这个场景特别适合本书前面讲过的所有核心能力：

- 需要工具获取信息
- 需要上下文和阶段状态
- 需要任务拆解

- 适合多智能体协作
- 结果可交付、可评估

这比只做一个“摘要 Agent”更完整，又没有大到失控，非常适合作为本书的系统级案例。

8.2 需求分析，先别急着搭



很多人一接到任务，最容易冲动上头，立刻开始写 prompt、接工具、想角色。但真正稳的项目，第一步一定是需求分析。

对于这个“内容研究与成稿系统”，我们先把需求拆成最原始的问题：

用户真正要什么

用户并不是只想要“一段文字”，而是想要一个可以代替大量人工前期工作的生产系统。这说明目标不只是输出内容，而是输出一个经过研究和整理的成稿结果。

系统最小可交付结果是什么

不是“所有事情都自动化”，而是能够稳定从主题出发，产出一篇结构清楚、信息支撑合理、可继续人工加工的文章初稿。

哪些步骤最关键

不是每一步都同等重要。在这个系统里，最关键的其实是：

- 资料质量
- 大纲结构
- 写作连贯性
- 最终审校

因为这四步决定了结果是不是像样，而不仅仅是“生成了一堆字”。

哪些步骤适合拆角色

这里已经能初步判断：

- 研究搜集适合单独拆
- 大纲规划适合单独拆
- 写作适合单独拆
- 审校适合单独拆
- 总体调度应由主控负责

到这里你会发现，系统其实已经开始长形了。这就是需求分析的价值。它不是多一道形式主义，而是帮你避免后面一边搭一边乱改。

8.3 任务拆解，把“大目标”拆成“稳定模块”



接下来，把任务正式拆成模块。

模块一，主题理解

负责理解用户到底要写什么，判断主题边界、读者对象和内容方向。

模块二，资料研究

负责外部信息获取，包括搜索、抓取、筛选、整理，形成初步材料池。

模块三，结构规划

负责根据主题与资料，生成内容框架和章节逻辑。

模块四，正文写作

负责基于已有大纲与资料，输出初稿内容。

模块五，质量审校

负责检查结构、逻辑、重复、事实支持和表达问题。

模块六，结果整合与交付

负责统一输出格式，给出最终成稿和必要说明。

你会发现，这种拆法有几个好处：

- 每个模块目标明确
- 每个模块输出可验证
- 模块之间可以清楚交接
- 后续方便增改和优化

这就是为什么复杂任务一定要先拆。不是为了显得专业，而是为了让系统变得有组织。

8.4 架构设计，把模块映射成 Agent



有了任务模块之后，就可以开始做 Agent 映射。

本案例可以设计成下面这个多智能体结构：

主控 Agent

负责接收用户任务、拆解流程、协调执行顺序、汇总最终结果。

研究 Agent

负责搜索和整理资料，输出研究摘要与来源清单。

规划 Agent

负责根据用户目标和研究资料，生成大纲与内容结构。

写作 Agent

负责根据大纲和资料生成正文。

审校 Agent

负责检查文章质量，提出修改建议并生成修订版。

这样的结构有几个明显优势：

- 每个 Agent 负担更专一
- 上下文更容易控制
- 问题更容易定位
- 后续扩展更方便

比如以后如果你想加“SEO 优化 Agent”或者“风格统一 Agent”，就能自然接进去，而不用把整个系统拆掉重来。

8.5 工具与记忆怎么接入



接下来进入最关键的落地层。

工具层

这个系统至少需要下面几类工具：

- 搜索工具，用于外部信息获取
- 网页抓取工具，用于读取页面内容
- 文档读写工具，用于保存中间结果和最终成稿
- 任务交接工具，用于 Agent 间传递结构化结果

如果做得更完整，还可以加：

- 术语表维护工具
- 格式化输出工具
- 版本对比工具

记忆层

这个系统的记忆不要做得太重，但要有层次：

短期记忆

- 当前主题
- 当前研究结果
- 当前大纲版本

- 当前草稿状态

长期记忆

- 用户偏好的写作风格
- 常用术语
- 文章模板
- 已验证的写作规范

这样一来，系统既能保持当前任务连续性，又不会因为一次任务的中间噪音污染整体长期行为。

8.6 Workflow 设计，把系统真正跑起来



把前面的东西串起来，这个案例的 Workflow 可以是：

1. 用户提交主题
2. 主控 Agent 解析任务
3. 研究 Agent 搜集资料并输出研究摘要
4. 规划 Agent 生成大纲
5. 写作 Agent 产出初稿
6. 审校 Agent 检查并修订
7. 主控 Agent 汇总交付

这是一个标准串行主链。

如果要优化效率，还可以把某些研究子任务并行化，比如：

- 一个 Agent 查背景
- 一个 Agent 查案例
- 一个 Agent 查数据
- 最后统一汇总

这样，整个系统就从一个概念图，变成了可以真正执行的流程链。

8.7 调试、修正与上线前检查



系统搭好不代表能交付。大多数 Agent 案例死就死在“能跑一次”和“能稳定跑很多次”之间。所以正式上线前，至少要检查这些问题：

1. 资料质量是否稳定

研究 Agent 拉回来的内容是否够相关、够可靠。

2. 交接格式是否统一

研究结果给规划 Agent，大纲给写作 Agent，格式是否稳定一致。

3. 写作是否过度发挥

正文是否基于资料，而不是自己乱扩。

4. 审校是否真正有效

审校 Agent 不是只会说几句空泛建议，而是真能发现结构和内容问题。

5. 整体时间与成本是否可接受

系统跑一篇文章如果太慢、太贵，实际业务也难落地。

8.8 复盘，这个案例真正说明了什么



这个案例最重要的，不是证明多智能体有多炫，而是让你看见一件事：

一个 Hermes Agent 系统，真正落地时，本质上是在做系统工程，而不是在堆 prompt。

你得会拆需求，会定边界，会选工具，会控上下文，会做角色分工，会做流程设计，还得会调试和复盘。真正的门槛，从来不是“会不会写一段提示词”，而是“能不能把很多能力拧成一个能跑的整体”。

本章小结



这一章，我们用“多智能体内容研究与成稿系统”完整走了一遍从需求到原型的落地路径。重点不是某个具体工具，而是如何把前面几章讲过的 Agent、Tool、Memory、Workflow 和 Multi-Agent 结构真正拼成系统。

如果前几章是在给你零件，这一章就是第一次带你真正组装整机。

延伸阅读



进入下一章前，可以继续带着这些问题：

- 系统能跑之后，成本怎么控
- 响应速度慢怎么办
- 失败重试和容错该怎么做
- 如何评估一个 Agent 系统到底是不是在变好

第九章 性能优化与稳定性治理



很多 Agent 系统在前期最容易掉进一个幻觉里：只要“能跑出来”就算成功。这在早期做 Demo 时还说得过去，但只要你真想把 Hermes Agent 往业务里推，这个标准就完全不够。

因为真实系统的评价标准从来不只是“能不能做”，而是：

- 做得稳不稳
- 做得快不快
- 做得值不值
- 出错时能不能收得住
- 用久了会不会越来越乱

这些问题，统统属于性能优化与稳定性治理的范畴。

换句话说，一个 Agent 系统从玩具走向产品，中间最大的分水岭之一，不是功能多少，而是你有没有开始认真治理它。

9.1 成本控制，不是小气，是系统理性



先说最现实的问题，成本。

很多 Agent Demo 在演示阶段都很爽，模型开最强、上下文塞满、工具调用随便跑、多个 Agent 轮番上阵。第一次看，确实很有冲击力。但真要长期跑起来，你很快就会发现，成本不是附属问题，而是系统结构问题。

因为成本本质上反映的是：你的系统有没有在用正确的方式解决问题。

如果一个简单任务也非要上最重模型、最长上下文、最多工具，那说明系统设计本身就有冗余。真正成熟的系统，不是为了省钱而省钱，而是知道不同任务该用什么配置，避免把所有问题都按最高规格硬砸。

通常来说，成本治理可以从这几层下手：

模型分层

不是所有环节都用同样的模型。规划、审校、路由这类任务，有时不需要最重模型。真正需要高推理强度的步骤，再给高配。

上下文压缩

能摘要就别全塞，能结构化就别拖长文本。上下文每多一点，都在持续烧成本。

工具调用节制

该不调的时候不调，该一次完成的不要重复调。很多系统成本高，不是模型贵，而是工具乱用导致流程冗长。

任务分级

高价值任务走高规格，低风险任务走轻量链路。别把所有请求都当成总统级接待。

9.2 响应速度，慢不是一定更稳



很多人会默认接受一个误区，觉得 Agent 系统复杂一点、慢一点很正常。正常是一回事，慢到影响体验和业务可用性，就是另一回事了。

Agent 慢，通常有几个来源：

- 上下文太长
- 工具调用太多
- 多智能体链路过深
- 串行过多
- 失败重试太频繁
- 每一步都用高成本模型

所以优化速度，本质上是优化系统路径。

第一，减少不必要步骤

如果一个任务三步够，就别设计成六步。流程越长，延迟越高，失误点也越多。

第二，能并行的别串行

尤其是信息搜集、子任务分析、资料整理这类独立性较高的环节，并行能明显提速。

第三，分清“必须实时”和“可以异步”

不是所有结果都必须一口气当场给完。有些复杂任务完全可以先返回阶段状态，再异步补全结果。

第四，别让 **Agent** 在简单任务上过度思考

简单任务该直接执行就直接执行。如果连轻量任务都走完整复杂链路，系统只会变得迟钝。

9.3 失败重试与异常恢复，决定系统有没有韧性

真实环境里，失败不是例外，是日常。

网页会超时，API 会报错，工具会失效，外部数据会异常，模型输出也会偏。所以成熟系统的关键，不是“尽量不失败”，而是“失败了怎么办”。

这就是为什么失败重试和异常恢复要被视为系统主能力，而不是补丁。

一个有韧性的 Hermes Agent 系统，至少要处理好三种异常：

工具级异常

比如接口报错、文件读不到、网页抓取失败。

应对方式通常是：

- 明确报错
- 合理重试
- 达到阈值后降级或终止

任务级异常

比如某一步产出质量太差，不足以进入下一步。

应对方式可以是：

- 退回上一层重做
- 请求补充输入
- 切换备用路径

系统级异常

比如多个 Agent 状态不同步、关键流程中断、结果汇总失败。

这类问题更偏架构层，需要在流程设计上预先考虑恢复点和回滚点。

一个系统真正稳，不是因为它不摔跤，而是因为它摔了之后爬得起来。

9.4 监控、日志与评估，没有这些就别谈优化



优化最怕瞎调。而瞎调的根源，往往就是系统没有日志、没有监控、没有评估。

你如果不知道问题出在哪，就只能碰运气。今天改 prompt，明天换模型，后天加工具，系统像算命一样调，最后自己都不知道到底哪次修改有效。

所以，Agent 系统只要开始认真做，就必须有最基本的可观察性。

日志

至少要能看到：

- 输入任务
- 每一步执行了什么
- 调用了哪些工具
- 返回了什么结果
- 哪一步失败

监控

至少要能看到：

- 响应时长
- 成功率
- 工具失败率
- 重试次数

- 成本变化

评估

至少要能回答：

- 结果质量有没有变好
- 稳定性有没有提升
- 用户反馈有没有改善
- 哪一步最拖后腿

没有这些，你就不是在做优化，你是在许愿。

9.5 用数据驱动迭代，不要靠感觉修系统



很多 Agent 项目之所以越做越飘，就是因为后期优化几乎全凭直觉。觉得结果不行，就改 prompt；觉得慢，就换模型；觉得输出乱，就再加一层 Agent。这套做法短期能碰巧解决一点问题，但长期一定失控。

真正成熟的治理方式一定是数据驱动：

- 哪类任务失败率最高
- 哪个步骤最慢
- 哪个工具最不稳定
- 哪类用户最常投诉
- 哪种输出结构最容易返工

这些信息一旦有了，你才能判断系统优化优先级。否则所有问题都会显得同样紧急，最后团队永远在忙，却很少真正进步。

本章小结



这一章的核心结论可以压成一句话：

Agent 系统能跑只是起点，能稳、能快、能控成本、能恢复，才算真正可用。

我们从成本、速度、异常恢复、日志监控和数据驱动迭代几个层面，讨论了 Hermes Agent 系统从 Demo 走向产品级可用所必须面对的治理问题。真正成熟的 Agent 工程，不只是能力堆叠，更是持续治理。

延伸阅读



带着下面这些问题进入最后一章会很有帮助：

- 一个系统从“自己能用”到“别人也能用”，中间差什么
- 如何做演示、交付和团队协作
- Agent 系统如何持续演进，而不是一次性交差
- Hermes Agent 的未来机会到底在哪

第十章 从可用到可交付



很多人做 Agent 系统，做到前面九章那一步，已经很有成就感了。系统能跑，流程能通，角色也拆了，工具也接了，日志也有了，看起来已经像回事。

但本王得泼点冷水。

能用，不等于能交付。

这是很多技术项目最后迟迟落不了地的根本原因。做出来的人觉得“已经挺牛了”，但真正拿到团队、客户、业务环境里，大家关心的根本不是你内部架构有多漂亮，而是：

- 这玩意我怎么用
- 它能稳定用多久
- 出问题谁来兜
- 能不能给别人讲清楚
- 能不能交给团队维护
- 后续怎么继续迭代

这就是从“可用”到“可交付”的那道坎。

如果说前几章解决的是系统如何做出来，那么这一章要解决的，就是系统如何被别人接住、理解、采用和持续演进。

10.1 演示，不是炫功能，而是讲清价值闭环



很多人做演示时，特别喜欢一股脑把所有功能都抖出来。能搜、能写、能调工具、能多 Agent 协作、还能记忆回溯，一套打下来像杂技表演。看着热闹，但业务方往往看完还是懵的：所以它到底帮我解决什么问题？

真正好的演示，不是功能清单展示，而是价值闭环展示。

也就是说，你要让别人看到：

1. 输入是什么
2. 系统做了什么
3. 输出带来了什么结果
4. 相比原来省了什么、快了什么、稳了什么

演示的重点不是“它能干多少事”，而是“它替你解决了哪个痛点”。

所以做演示时，最稳的方式不是铺全，而是选一个最能代表价值的闭环案例，走完整一遍。

10.2 交付，不只是给代码，更是给使用方式



很多技术团队把交付理解成“把系统部署好，代码给到，文档扔过去”。这不叫交付，这顶多叫移交文件。

真正可交付的 Hermes Agent 系统，至少要让接手的人知道四件事：

- 它能做什么
- 它不能做什么
- 正确使用方式是什么
- 出问题时怎么排查

所以交付最少应该包括：

使用说明

任务如何输入，结果如何查看，哪些参数可调，哪些行为是默认的。

边界说明

什么场景适合，什么场景不适合，哪些结果需要人工复核。

故障说明

常见失败模式、常见异常提示、如何做基础排查。

版本说明

当前系统包含哪些能力，哪些还没做，哪些是下一版本规划。

你会发现，这其实已经不是纯技术动作，而是产品化动作。真正能交付的系统，一定不是只有开发者自己会用的系统。

10.3 团队协作里，Agent 系统也需要分工



当 Hermes Agent 系统进入团队使用阶段，问题会突然从“技术怎么做”变成“组织怎么协同”。

这时候至少会出现几类角色：

- 做系统设计的人
- 写 Prompt / 规则的人
- 接工具的人
- 做评估和调试的人
- 使用系统的人
- 给结果兜底的人

如果这些角色关系不清，系统即便做出来，也会在协作中越来越乱。比如出了问题，到底怪 prompt、怪模型、怪工具、怪数据，还是怪业务方输入不清？没有清晰分工，大家会互相甩锅，系统就会停在“能跑，但没人敢负责”的状态。

所以团队协作阶段，很重要的一件事，就是明确责任边界。谁负责系统骨架，谁负责能力扩展，谁负责质量评估，谁负责最终验收，必须尽量清楚。

10.4 持续迭代，别把 Agent 系统做成一次性交付品

很多人做 Agent，心态还停留在传统项目交差模式：做完、交付、结束。但 Agent 系统几乎从来都不是一次性工程。

原因很简单：

- 模型会变
- 工具会变
- 业务会变
- 用户预期会变
- 任务分布会变

也就是说，一个 Hermes Agent 系统如果真进入使用阶段，它天然就应该被看作持续迭代产品，而不是一次性交差物。

所以你最好从一开始就接受这个现实：

- 第一版只求跑通
- 第二版开始控质量
- 第三版开始控成本和效率
- 后续版本不断针对高频问题做治理

真正成熟的系统，不是第一版就完美，而是有演进路径。

10.5 Hermes Agent 的未来机会



最后这一节，不想写成那种空洞的行业鸡汤。我们就实打实说几件事。

Hermes Agent 的机会，不在于它是不是一个新潮名字，而在于它代表的这类系统能力，正好卡在一个非常现实的产业需求上：

大家已经不再满足于“AI 很聪明”，而是开始要求“AI 能不能稳定替我干活”。

只要这个需求成立，围绕 Agent 的落地问题就一定会持续存在：

- 怎么搭
- 怎么控
- 怎么协作
- 怎么交付
- 怎么治理
- 怎么规模化

这意味着，未来真正值钱的，不只是会用某个模型的人，而是能把模型、工具、流程、记忆和团队协作方式拼成稳定系统的人。

这也是本书整本都在强调的一件事：

Agent 的竞争力，不在嘴上，在系统里。

你最终拼的不是“谁知道更多术语”，而是谁能把一个复杂任务设计成稳定的执行系统，谁能让 AI 从 Demo 变成生产力。

本章小结



这一章最核心的结论可以压成一句话：

一个 Hermes Agent 系统，只有当别人也能理解、使用、接手和迭代时，才算真正完成了从可用到可交付的跨越。

我们讨论了演示、交付、团队协作、持续迭代与未来机会。说到底，这一章讲的不是某个技术细节，而是一个现实判断：真正能活下来的 Agent 系统，一定不是只存在于开发者电脑里的系统，而是能进入组织、进入流程、进入业务、并被持续运营的系统。

附录 A 常见术语表



Agent 围绕目标持续推进任务的 AI 执行体。

Tool Agent 用于接触外部世界的能力接口，如搜索、读写文件、调用 API 等。

Memory 为 Agent 提供跨步骤或跨任务连续性的信息存储与检索机制。

Workflow 任务的执行路径与步骤组织方式。

System Prompt 定义 Agent 角色、职责、边界和底层行为原则的系统级提示。

Short-term Memory 服务当前任务过程的临时状态记忆。

Long-term Memory 服务跨任务复用的稳定信息记忆，如偏好、规则、项目背景等。

Multi-Agent 多个 Agent 基于分工协同完成复杂任务的系统结构。

Routing 根据任务类型或状态，将任务分发到合适 Agent 的过程。

Context Window 模型当前轮次可以直接看到的输入信息范围。

Observability 系统可观察性，包括日志、监控、状态追踪和错误定位能力。

Fallback 当主要路径失败时，系统进入的备用或降级处理路径。

附录 B Hermes Agent 常见问题排查清单



结果总跑偏

- 任务目标是否写清
- 输出格式是否明确
- System Prompt 是否职责太泛
- 是否让 Agent 同时做了太多种类型任务

工具乱调用

- 是否给了太多工具
- 是否没有写清工具调用时机
- 是否没有规定“不需要时不要调用”

用久了越来越乱

- 是否把临时信息错误写进长期记忆
- 是否没有记忆回收机制
- 是否所有 Agent 共用过多上下文

成本过高

- 是否所有步骤都在用高规格模型
- 是否上下文太长

- 是否存在重复工具调用
- 是否流程设计过深

系统太慢

- 是否本可并行却全在串行
- 是否任务拆分过度
- 是否简单任务也走了复杂链路

多智能体互相打架

- 角色边界是否不清
- 交接格式是否混乱
- 是否没有统一汇总与仲裁层

附录 C 推荐实践清单



1. 先做最小闭环，再做复杂能力
2. 角色定义尽量具体，不要空泛
3. 工具只接当前任务真正必要的
4. 记忆不是越多越好，重点是筛选与回收
5. 多智能体不是目的，稳定分工才是目的
6. 先保证可观察性，再谈优化
7. 出错不可怕，怕的是错误不可见
8. 先交付一个可用版本，再持续迭代
9. 不要试图用 prompt 修复所有架构问题
10. 真正值钱的是把 AI 变成稳定系统的能力

全书结语



如果你一路看到这里，你应该已经能感觉到，本书真正想讲的，从来不只是 Hermes Agent 这个名字本身。

它想讲的是一个更底层、也更现实的能力转变：

我们正在从“如何使用模型”，走向“如何设计系统”。

过去大家比的是谁更会写提示词，谁更会找模型，谁更快摸到新工具。但接下来真正拉开差距的，一定是另一种能力：

谁能把模型、工具、记忆、流程、角色和治理方式真正组织起来，做成一个能持续运行、能稳定交付、能不断演进的系统。

这就是 Agent 工程真正有意思的地方。它不只是 AI 技术应用的一小支，而是在逼着我们重新理解一件事：

智能，只有被组织进系统，才会变成生产力。

Hermes Agent 只是你进入这个世界的一扇门。门后真正重要的，不是某个框架、某个 prompt、某个技巧，而是你是否开始具备了系统思维，是否能把一个模糊任务设计成可执行结构，是否能让 AI 在现实中稳定干活，而不是只会在演示里发光。

如果这本书真的帮到了你，本王希望你最后带走的，不只是几个术语或几套模板，而是下面这句话：

不要只做会调用 AI 的人，要做会组织 AI 系统的人。

这才是更长远的硬本事。